CS 454 Theory of Computation Fall 2025

Project 1, Due: Oct 15, 2025

The project will be done in teams of two members. Just one submission is required per team. In addition to the code, please submit a short report that *states clearly the contribution of each member of the team.* Submission details will be specified in detail (along with test cases to be included with the submission) in the canvas submission page. The project will be graded on (a) correctness on the test cases, (b) meeting the user interface requirements and (c) following the guidelines (described at the end).

This updated version contains a more detailed description of the algorithm to solve Problem 2.

The goal of the project is to compute the number of strings w of length n over $\{a, b, c, d\}$ with the following property: In any substring of length 6 of w, all three letters a, b, c and d occur at least once. For example, strings like *abbccdaabca* satisfy the property, but a string like *bad***aabcbc***dcabad* does not satisfy the property since the substring **aabcbc** does not have a d. We also want to count the number of strings of even length that, in addition to the above condition, also has aa as the two middle symbols. For example, *bcad***aa***bcb* is one such string.

The idea is to create a DFA M for the language:

$L = \{w \mid$ in any substring of length 6 of w, all the letters a, b, c and d occur$\}$.

By definition, all strings of length less than 6 are in L.

The idea behind an efficient solution to this problem is to design a DFA for L and to use an efficient algorithm to compute the number of strings of length n accepted by a DFA.

The algorithm for the counting the number of strings of a given length accepted by a DFA is briefly presented below:

Let $M = \langle Q, \Sigma, \delta, 0, F\rangle$ be a DFA. Assume that $Q = \{0, 1, \cdots, m-1\}$ and that 0 is the start state for an appropriate m. The first step is to construct this DFA. This DFA around 1400 states so the construction of the δ function requires a program. The states of the DFA will encode a buffer that holds the last 5 symbols scanned. When the next input symbol is read, the DFA checks if the last 6 symbols scanned (the five from the buffer together with the input symbol read) meets the requirement that all the letters occur at least once. If this condition is not met, the DFA enters a fail state and thereafter it will remain there. If the condition is met, then the buffer is updated by dropping the leftmost symbol of the buffer and adding the scanned symbol. Initially the buffer is empty (represented by ϵ). The five transitions will fill the buffer. Formally, if the current state q encodes a string $b_1b_2\cdots b_m$ where $m < 5$, then $\delta(q, a)$ encodes the state $b_1b_2\cdots b_ma$. If the state p encodes a buffer of length 5, say $b_1b_2b_3b_4b_5$, then $\delta(q, a)$ encodes the state $b_2b_3b_4b_5a$ if $b_1b_2b_3b_4b_5a$ contains at least once occurrence of each a, b, c and d, else $\delta(q, a)$ is the fail state. For the problem to be solved, it is convenient to map the buffer string to a positive integer so that the states can be labeled 0, 1, 2, etc. One such encoding is to use base-4. For example, *babca* represents the integer $2 \times 4^4 + 4^3 + 2 \times 4^2 + 3 \times 4^1 + 4^0$.

Next we define a subset of L, namely the language $L' = \{w \mid w$ is in L, $|w|$ is even and the middle two characters of w are $aa\}$. L' is not regular but we can build a DFA for a related language L'' such that the number of strings of length $2m$ in L' is equal to the number of strings of length $m-1$ in L''. The description of L'' and a DFA for L'' are described below:

We first consider a related, but slightly different, problem. Suppose L_1 is a regular language over an alphabet Σ. Define L_2 over $\Sigma \times \Sigma$ as follows.

$L_2 = \{(a_1, b_1)(a_2, b_2) \cdots (a_k, b_k) \mid a_1 a_2 \cdots a_k b_1 b_2 \cdots b_k \in L_1\}$.

We will show that L_2 is regular as follows: Let $M_1 = \langle Q, \Sigma, \delta, 0, F\rangle$ be a DFA for L_1. For a fixed $p \in Q$, we first define a DFA M_p is described below.

$M_p = \langle Q \times Q, \Sigma \times \Sigma, \delta_1, (0, p), F_1\rangle$ where $\delta_1((q_1, q_2), (a_1, a_2)) = (\delta(q_1, a_1), \delta(q_2, a_2))$, $F = \{(p, f) \mid f \in F\}$. It can be observed that $L(M_0) \cup L(M_1) \cup \cdots = L_2$. A standard way to build a DFA for L_2 will be to create a starting state s and have ϵ transition from s to the starting state of M_p for each p. This NFA turns out to be an unambiguous NFA so for the purpose of counting the number of strings in L_2, we can simply count the number of strings (of a given length) in each M_p and add them.

Getting back to Problem 2 of the project, the above construction can be used with a small modification. In the above construction, the mapping between strings in L_1 and L_2 is: $(a_1, b_1)(a_2, b_2) \cdots (a_n, b_n) \equiv a_1 a_2 \cdots a_n b_1 b_2 \cdots b_n$. In Problem 2, the mapping is slightly different: $(a_1, b_1)(a_2, b_2) \cdots (a_n, b_n) \equiv a_1 a_2 \cdots a_n \mathbf{aa} b_1 b_2 \cdots b_n$. To accommodate this change, the only change we need to make in the DFA M_p is to make the start state as $(0, q)$ where $q = \delta(p, aa)$. This now provides a one-to-one mapping between a string $a_1 a_2 \cdots a_n aa b_1 b_2 \cdots b_n$ and $(a_1, b_1)(a_2, b_2) \cdots (a_n, b_n)$.

The next step is to implement a program that takes as input a DFA M and an integer n and computes the number of strings of length n accepted by M. The algorithm for this counting problem was discussed in class. Specifically, the algorithm computes $N_j(n)$ = the number of strings of length n starting from state j and reaching an accepting state.

The number of strings of length n accepted by a DFA M is given by $N_0(n)$ since 0 is the starting state. The recurrence formula for $N_j(n)$ is given as follows: $N_j(n) = \Sigma_{x \in \{a,b,c,d\}} N_{\delta(j,x)}(n-1)$. Initial values $N_j(0)$ are given by: $N_j(0) = 1$ if $j \in F$, $N_j(0) = 0$ if $j \notin F$. Using this recurrence formula, you can compute $N_j(k)$ for all j for $k = 0, 1, \cdots, n$. As we noted in class, you only need to keep two vectors *prev* and *next* of length m where m is the number of states. Using the values of $N_j(k)$ stored in *prev*, you can compute $N_j(k+1)$ for all $0 \leq j \leq m-1$ in *next*. Then copy *next* to *prev* and repeat.

When your main function is run, it will give choice (1) for counting the number of strings of a given length n in L, (2) for counting the number of strings of a given length n in L', (3) to quit. The choices (1) or (2) will be in a loop until (3) is entered when the program halts.

The range of n will be between 1 and 300 for both problems. The answer should be exact, not a floating-point approximation so you should use a language that supports unlimited precision arithmetic like Java or Python or a library like GMP (in case of C++).

Some test cases will be provided soon.

Your code shall follow the directions provided above, namely it will create the DFA for the languages L and L'' with states labeled $\{0, 1, \cdots, t\}$ (t is around 1400 for L and around 1400^2 for L'' and the input symbols will also be labeled $0, 1, \cdots, t$ where $t = 3$ for L (15 for L''). The counting algorithm shall follow the steps described above in that it uses two arrays of size N where N is the number of states in the DFA to update the count in each iteration and copy back to the original array. For each function you implement, add comments to state clearly what the inputs and outputs are (with at least one example) and pre-conditions, if any, that should be satisfied.